

* Spring Boot Configuration:-

Date.....

* Problem with Hardcoding →

In real-world applications, hardcoding values (like database URLs, passwords, port no.) directly into Java code is a major anti-pattern.

Why?

- Environment Specific :- Code behaves differently in Local, Dev, Staging and Production environments. Hardcoding forces you to manually change and recompile code.
- Security Risks :- Sensitive information (credentials) gets committed to source control (like GitHub), leading to potential leaks.
- Lack of Flexibility :- Changing a configuration requires a full deployment cycle.

Solution :-

Use externalized configuration files like "application.properties" or "application.yml" to separate business logic from environment-specific parameters.

Date.....

Both of them work same, but only difference is 'YAML' uses indentation to represent nested objects whereas 'properties' uses a dot notation.

But, if both are present, Spring Boot will give priority to [.properties].

* Spring Profiles :-

Profiles allow you to define environment-specific configurations by creating multiple files with the naming convention.

→ How it Works :-

- Base Config → "application.properties" acts as the default, common configuration for all environments.
- Profile-Specific Files → You can create files like [application-dev.properties], [application-staging.properties] and [application-prod.properties] to override or add specific settings for those environments.

Date.....

Sagar 2/4

- Activation → To tell which profile to use, you set the property in your main [app.properties] file.

```
{ spring.profiles.active = dev }
```

When a specific profile is active, Spring merges settings from common file with settings in active-profile-specific ones.

→ External Configuration :-

To avoid hardcoding values or manually changing files during deployment, Spring boot allows you to provide configurations externally.

- Command Line Arguments → You can override properties at runtime using the command line. This is highly effective for CI/CD pipelines.

```
{ mvn spring-boot:run -Dspring-boot.run.profiles=prod }
```

- Environment Variables → You can set configuration values directly in the host Operating System or container environment (e.g. Docker), which Spring Boot will pick up automatically.

→ Why this matters?

This approach ensures your 'jar' file remains unchanged across different environments. You deploy the same binary to every environment and simply pass the profile (like dev or prod) or specific configuration variables as runtime arguments.

* Conditional Bean Creation:-

Beyond just configuration values, Spring allows you to conditionally register Spring Beans based on the active env. using "@Profile" annotation.

→ How it works?

- Interface Driven Design → Define a common interface (e.g. NotificationService) and create multiple concrete implementations.

Date.....

- Conditional Injection → Use `@Profile("dev")` on a dummy implementation and `@Profile("prod")` on the real.
- Selective Loading → Only the bean that matches the currently active profile is created by IOC Containers.

→ Key Differences :-

- Profile-Specific Files (`application-{env}.prop`) are used to vary configuration values (ports, URLs, keys).
- `@Profile` annotation is used to vary application behaviours / logic by controlling which classes are instantiated as beans.